

Retrieval Augmented Generation for Document Question Answering

NLP Lab Experiment Report

Abstract

This report documents a laboratory experiment that implements a Retrieval Augmented Generation (RAG) system. The system answers natural language questions using only the documents a user uploads, and every answer is accompanied by a citation pointing back to the exact passage it was generated from.

The experiment covers the full pipeline: text extraction, cleaning, chunking, embedding generation, vector storage, similarity retrieval and grounded answer generation.

1. Objective

The objective of the experiment is to build a question answering system that can answer questions about a personal document collection without any prior fine-tuning of a language model.

A secondary objective is to verify that answers are grounded: the system must refuse to answer when the retrieved context does not support an answer, rather than hallucinating one.

The objective of the experiment is therefore twofold: demonstrate the mechanics of retrieval augmented generation, and demonstrate that its outputs are traceable to their sources.

2. System Architecture

The architecture has six stages. First the uploaded file is parsed into plain text. The text is then cleaned to remove repeated page headers, fix hyphenation and normalise whitespace.

The cleaned text is split into chunks of 500 characters with an overlap of 100 characters. The overlap exists so that a sentence straddling a chunk boundary is still fully contained in one chunk.

Each chunk is encoded into a 384 dimensional vector. The vectors are stored in a ChromaDB collection configured for cosine similarity.

3. Embeddings

The embedding model is sentence-transformers/all-MiniLM-L6-v2. It is a small bi-encoder that maps a chunk of text into a dense vector of 384 dimensions, and it is chosen because it is fast on CPU while still capturing semantic similarity well enough for passage retrieval.

The same model embeds the user's question, which makes query and document vectors directly comparable with a cosine similarity.

4. Chunking Strategy

The chunking strategy uses 500 character chunks with an overlap of 100 characters. A chunk size of 500 characters keeps enough context for a meaningful passage while staying small enough that the embedding is not diluted by unrelated text.

The overlap of 100 characters is roughly one fifth of a chunk. This prevents a fact that falls exactly on a boundary from being split across two chunks, where neither half would retrieve it well.

5. Vector Database

The vector database used for storage is ChromaDB. It persists an HNSW index on disk, which makes cosine similarity search fast even as the collection grows, and it keeps the collection metadata alongside each vector so that citations can be resolved.

Each vector carries metadata recording the source file name, the chunk number and the page number. This is what makes it possible to show the user exactly which passage produced an answer.

6. Retrieval

Retrieval works by embedding the question with the same model used for the documents, then performing a similarity search that returns the top 5 chunks ranked by cosine similarity.

A similarity score close to 1 means the chunk means almost the same thing as the question, while a score near 0 means the two are unrelated. The interface displays these scores so retrieval quality can be inspected directly.

7. Answer Generation

Answer generation sends the retrieved context and the question to a large language model. The prompt instructs the model to answer only from the provided context and to reply that the documents do not contain enough information when the context is insufficient.

Because generation is conditioned on retrieved text, the system cannot invent facts about documents it has never seen. The cost of this design is that answer quality is bounded by retrieval quality.

8. Advantages of RAG

The advantages of RAG include grounded answers that can be traced to a source, knowledge that can be updated by editing documents rather than retraining a model, and transparency because the retrieved passages can be shown to the user.

RAG also reduces hallucination compared with asking a model directly, since the model is instructed to rely on the supplied context.

9. Limitations

A limitation of the system is that retrieval quality depends heavily on the chunk size: chunks that are too small lose context, while chunks that are too large dilute the signal and reduce precision.

Another limitation is that the system performs lexical and semantic matching only. It cannot answer questions that require arithmetic over the documents, or reasoning that spans many chunks at once.

Finally, scanned PDFs without a text layer cannot be processed because optical character recognition is out of scope for this experiment.

10. Conclusion

The experiment demonstrates that a complete retrieval augmented generation workflow can be built from standard open source components and run locally on a laptop CPU.

The key takeaways are that grounding depends on retrieval, that citations make the system auditable, and that abstaining is a feature rather than a failure when the corpus does not cover the question.